“龙虾”也需要看病？一张旧病历，引出三个新确诊

原创 腾讯程序员 腾讯技术工程 2026年4月10日 02:35 广东 199人



作者：悟空代码安全团队

> 本文是悟空Agent泛化能力系列的先导篇。我们不展开技术细节，只回答一个核心问题：为什么悟空Agent在漏洞挖掘中需要泛化能力。后续系列将深入拆解泛化体系的每个环节——感知、建模、自迭代、规模化验证。如果你对AI代码安全感兴趣，这是一个值得从头跟下去的故事。
>
> 👉 文末提供悟空 Agent 试用申请链接

一、AI找漏洞，已经超出所有人的预期

Anthropic 旗下的 Claude，在参与 FreeBSD 操作系统安全测试期间，不仅发现了一个此前从未被注意到的安全漏洞，更成功生成了一份可直接复现的漏洞利用代码。

Claude 所展示的，是 AI 在漏洞挖掘上的**深度**——能够在特定目标上持续深挖，找到人类研究者尚未触及的角落。然而，还有另一个维度同样关键，甚至更具挑战性：**广度**——发现一个漏洞之后，AI 还能顺藤摸瓜，找到什么？

这正是悟空 Agent 一直试图回答的问题。而在 OpenClaw（我们熟知的"龙虾"）上，我们得到了三个具体的答案。

二、三份报告之前：先聊聊"为什么找一个还不够"

设想一位经验丰富的传染病科医生，接诊了一例罕见病例。普通医生的处置逻辑是：**确诊→治疗→结案**。而有经验的医生则会多思考一层：这种疾病的发病机制究竟是什么？还有多少人带有相似症状，却尚未得到诊断？

安全领域的传统做法，更接近前者：**发现漏洞→修复→关闭工单→进入下一个目标**。鲜有人追问：这个漏洞的"同族"潜伏在何处？

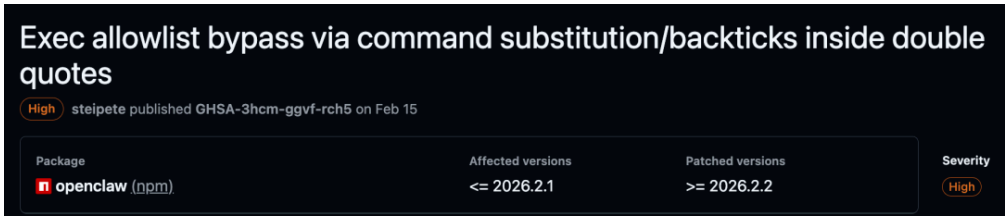
悟空 Agent 想做的，是后者——以一个已知漏洞为起点，推断"同族漏洞"可能存在的位置：既包括高度相似的直接变种，也包括表面上毫无关联、却在底层机制上一脉相承的"远亲"。

我们将这种能力称为泛化。发现一个漏洞，固然重要；但更有价值的，在于由此出发，系统性地识别并消除一整类同源漏洞——这才是泛化能力的意义所在。

三、案例实录：悟空Agent在 Openclaw 的三次"确诊"

本文介绍的三个漏洞，全部由悟空 Agent 检测，已向 Openclaw 官方正式提交并确认(最新版本已修复)。它们有一个共同的起点——同一张"病历"。

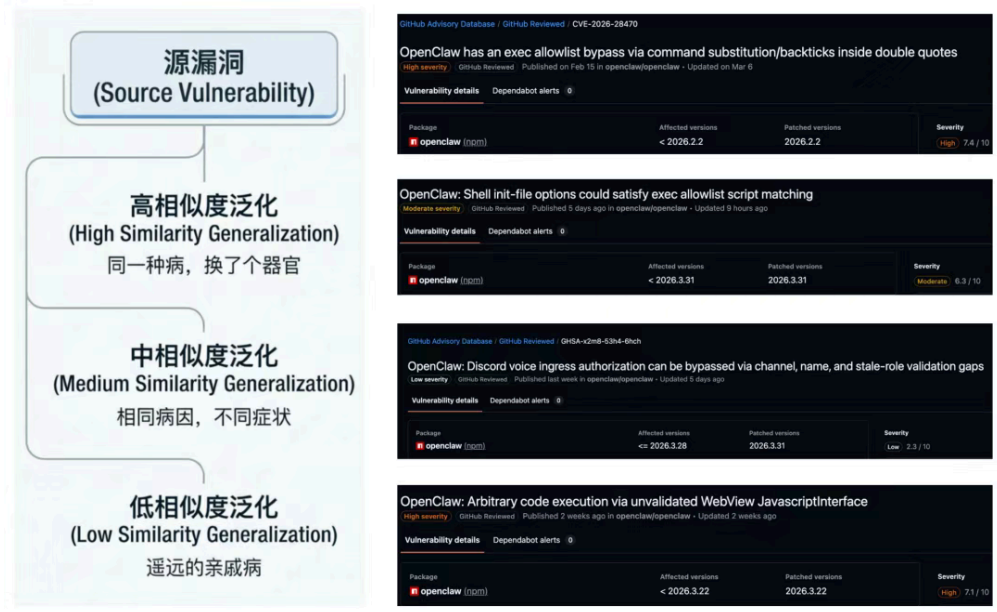
源头漏洞：GHSA-3hcm-ggvf-rch5



这个漏洞的核心很简单：攻击者可以在一条命令的双引号里藏入特殊语法（ $() 或反引号），系统的白名单检查看到的是一条"合规命令"，但 shell 在真正执行时，悄悄运行了藏在里面的另一套指令。

一句话：检查时看到的是安全的命令，但真正运行时却执行了被隐藏的额外指令。

悟空Agent从这张病历出发，用三种不同"松紧度"的泛化策略，找到了三个新漏洞。



Case 1：高相似度泛化——“同一种病，换了个器官”

源漏洞： GHSA-3hcm-ggvf-rch5（双引号内命令替换绕过exec白名单）

泛化漏洞： GHSA-wpc6-37g7-8q4w（Shell init-file选项绕过exec白名单）

● **危害类比：** 同一种肿瘤，第一次长在了胃部，第二次长在了肠壁——病理科报告显示，两个肿块的细胞结构高度一致，起源相同。如果只治了胃，没有对整个消化道做系统筛查，肠壁上的那个就会被漏掉。

● **悟空Agent 执行的泛化策略：**

○ 策略简述：「基于 GHSA-3hcm-ggvf-rch5，在当前仓库的 exec 白名单匹配逻辑中，逐一扫描所有 shell 调用形态，检查是否存在其他可以在"审查阶段"与"执行阶段"之间插入差异的参数或选项。

○ 约束程度： ★★★ 紧（同一模块、同一攻击面、精确复现逻辑）

● **泛化路径极短：** 原始漏洞 → 相同逻辑/相近代码结构 → 发现变种漏洞

○ 同在 exec 白名单机制内，只是从"双引号内的命令替换"扩展到了"shell wrapper的init-file参数"，代码相邻，模式高度重合。

● **说明：** 高相似度泛化是"最近的亲戚"，传统工具做代码 diff 也有机会发现，但悟空Agent能把"审查与执行之间存在差异"这个抽象模式转化为对所有 shell 参数的系统性扫描，速度和覆盖度远超人工。

Case 2：中相似度泛化——“相同病因，不同症状”

源漏洞： GHSA-3hcm-ggvf-rch5（exec 执行层：校验看到的命令，与最终执行的命令，不是同一件事）

泛化漏洞： GHSA-x2m8-53h4-6hch（ingress 授权层：校验时依赖的角色状态，与执行时的真实状态，不是同一件事）

● **危害类比：** 同一种细菌感染，一个引发肺炎，一个引发肠炎——病因相同，但如果没有系统性思维，两个病例可能永远不会被关联起来

● **悟空Agent 执行的泛化策略：**

○ 「基于 GHSA-3hcm-ggvf-rch5 揭示的"校验对象与执行对象不一致"这一根因，不局限于 exec 命令执行层，扫描整个系统中所有存在"校验时状态"与"执行时状态"可能发生偏移的授权逻辑，检查这些偏移是否可被利用。」

○ 约束程度： ★★ 中（跨模块，攻击面从命令执行扩展到通信层授权）

● **泛化路径：** 从漏洞触发模式推演到不同上下文、不同模块中的类似问题

○ 这里需要从"命令解析被欺骗"这个绕过策略，抽象出"校验层与执行层之间的状态不一致"这一更通用的漏洞模型，再将其投影到 ingress 的授权逻辑上。这是语义跳跃，传统工具基本无法做到。

● **说明：** 这里体现的是语义理解能力，而非简单的代码模式匹配，传统工具基本无法覆盖

Case 3：低相似度泛化——“遥远的亲戚病”

源漏洞： GHSA-3hcm-ggvf-rch5（外部输入越过了信任边界，系统没有完整解析它"是什么"）

发现漏洞： GHSA-cxmw-p77q-wchg(外部输入越过了信任边界，系统没有验证它"来自哪里"）

● **危害类比：** 医生研究某糖尿病患者的发病机制，意外发现同一套底层逻辑能解释一个完全不同科室的自免疫疾病——表面毫无关联，底层机制相通

● **悟空Agent 执行的泛化策略：**

○ 「基于 GHSA-3hcm-ggvf-rch5，不局限于 CLI/命令执行层，不局限于桌面端或服务端，将"外部输入在进入高权限执行上下文之前，系统是否完整校验了它是什么、以及它来自哪里"作为核心检测命题，在整个代码仓库所有平台的信任边界处展开系统性排查。」

○ 约束程度： ★ 宽（跨平台、跨技术栈、跨攻击面，仅保留最底层的根因模型）

● **泛化路径：** 跨模块、跨上下文的深层语义关联推断

● **说明：** 这是泛化能力真正的"天花板"。它不再问"同样的代码在哪里出现过"，而是问"同样的认知盲区在整个系统里还有哪些角落"。这种抽象层级的迁移，是大模型区别于一切传统工具的本质能力所在。

回顾这三个案例，不难发现它们并非处于同一层次。从高度相似的代码变种，到机制相通却表现迥异的中间形态，再到仅凭底层逻辑关联推断出的"远亲"漏洞，泛化的难度依次递进，所需的推理跨度也截然不同。我们将上述三个案例归纳为三个泛化等级——

泛化等级	通俗类比	策略约束	技术难度	传统工具能做到吗?
高相似度	同一种病，换个器官	★★★紧	★☆☆	部分可以
中相似度	同病因，不同症状	★★☆中	★★☆	需要大量人工介入
低相似度	遥远的亲戚病	★☆☆宽	★★★	超出工具能力边界

四、 悟空 Agent 关于泛化 Harness 的实践

大语言模型本身已具备泛化能力，但为何实际应用中的泛化表现依然不稳定？答案往往不在模型，而在 Harness 的设计。以下是我们在悟空 Agent 实践中总结的四条核心原则。

原则一：给起点，不给终点；给根因，不给结论

直接告知模型推断结论，会压缩其推理空间，使其退化为执行者。正确的做法是只提供根因描述，同时给出一个推理锚点（代码片段、攻击模式或信任边界定义），让模型自主走完中间的推理路径。锚的抽象层级决定泛化深度：高相似度泛化以代码为锚，低相似度泛化以语义为锚。

原则二：约束松紧度是可以主动调节的设计参数

约束过紧，覆盖面窄，容易遗漏远亲类漏洞；约束过松，推理发散，噪音激增。我们的做法是分层触发——针对同一源漏洞，分别以高、中、低三种约束程度独立运行，再对结果进行置信度过滤汇总。

原则三：将推理过程显式化

要求模型在输出结论前先呈现推理链（根因 → 推断位置 → 理由），能显著提升结论质量，也让人工审核更高效。如果忽略推理过程，人工审核往往会无法关联上两个“远亲”漏洞。

原则四：用新发现持续反哺推理起点

每一次泛化所发现的新漏洞，都不应仅作为独立结果处理，而应反哺至下一轮泛化的输入——更新锚点，修正根因描述，丰富已有的漏洞模式库。这使 Agent 的泛化能力持续进化，而非停滞于初始状态。

这四条原则同样适用于其他需要"从已知案例推断同类问题"的 Agent 场景。大模型的泛化能力是内置的，Harness 的作用，是为它划定合适的推理舞台。

五、为什么这件事很重要：攻击者只需赢一次，防御者必须次次不败

这场博弈，规则本就不对等。

对攻击者而言，找到一个漏洞便已足够——一个入口，一次突破，全盘皆输。而对防御者而言，修掉一个漏洞几乎什么都不代表：只要还有第二个、第三个尚未被发现的缺口，系统就依然暴露在风险之中。**防守方真正需要的，是将所有漏洞、所有变种悉数找出并逐一补上，方能称得上真正意义上的安全——这正是我们研究泛化能力的根本出发点。**

传统工具擅长识别已知威胁，却对"它的变种藏在何处"束手无策。当下已有不少团队开始将 AI 引入漏洞挖掘，这无疑是正确的方向。然而，现阶段多数探索仍聚焦于提升单点漏洞的发现效果——这是 AI 在安全领域落地的自然起点，也已取得了切实的进展。

悟空的泛化能力，试图做的是另一件事——从"发现一个漏洞"出发，将其家族、其变种、其在整个系统中所有可能的表现形式，一次性摆上桌面。不是让防守方跑得与攻击者一样快，而是换一种打法：溯源根因，而非逐一追着症状奔跑。

这不是 AI 能力的技术展示，而是防守方在这场天然不对等的博弈中，所能找到的最务实的一条出路。

六、 下一步：悟空Agent 即将开放外部试用

这篇文章是悟空Agent泛化能力系列的先导篇，我们接下来会逐一拆解这套泛化体系的每个环节。

如果你对这个系列感兴趣，可以先留下联系方式——4月底，悟空Agent 将开放外部试用，第一批名额优先向预约用户开放。



· 📋 预约试用入口：悟空AI挖洞申请试用 – 腾讯问卷

我们在等你一起来找"亲戚病"。

关于腾讯悟空代码安全团队

腾讯悟空代码安全团队（WuKong, Tencent）隶属腾讯安全平台部，专注于 AI 驱动的代码层漏洞检测与安全风险治理。自研悟空代码安全 Agent 已在 GitHub 知名开源项目中累计发现并确认 400+ 0day 漏洞，获微软、英伟达、Apache 等知名企业与开源组织致谢。团队曾发布业内首个项目级 AI 生成代码安全评测集 A.S.E，团队成员过往研究成果发表于 S&P，USNEIX Security，FSE，ASE，ICSE，ACL 等国际顶尖学术会议。





悟空 AI Agent · 目录

上一篇
首发！建议你一定要看的《AI 生成代码在野安全风险研究》

下一篇
从3632个漏洞看AI时代的评测基准重构！VulnGym基准发布

阅读 1.1万

留言

写留言

还没有留言，来写首评